

推荐PM手把手入门第25讲：本地生活策略，LBS、即时供给与地理栅栏的逻辑

👉 上一讲聊完电商推荐，有朋友问我："本地生活的推荐是不是更简单？不就是看谁离得近推谁吗？" 说实话，我一开始也这么想过。但当我真正去研究美团、饿了么这些平台的推荐策略时，发现这里的门道比想象中复杂太多——地理位置不是一个简单的距离计算，而是一整套需要精细设计的分层体系。

今天这一讲，我想从PM视角聊聊本地生活推荐的三个核心模块：LBS、即时供给，以及地理栅栏。这些概念听起来很技术，但理解它们的产品逻辑后，你会发现这才是本地生活推荐的真正壁垒所在。

LBS不是简单的"距离计算"，而是分层的地理策略

先说个很多人的误区：觉得本地生活推荐就是"算距离，推最近的"。但实际上，如果真这么做，你会发现推荐效果非常糟糕。为什么？因为"近"这个概念本身就很模糊。用户在北京三里屯想吃火锅，你推荐500米外的一家小店，还是推荐2公里外的海底捞？

这里的核心思路是：**LBS需要分层设计，不同场景用不同的地理粒度。**

我观察下来，主流平台基本都会分成三层。



城市级：新城市冷启动的基础

城市级的粒度是最粗的，覆盖整个城市范围。这一层主要解决的是冷启动问题——当平台刚进入一个新城市时，用户和商家数据都很少，这时候你不可能做精细化推荐，只能先按城市整体运营。

举个例子，美团刚进入一个三线城市，可能整个城市只有200家合作商家，这时候谈"附近3公里"没意义，因为用户附近可能就没几家店。这时候策略就是：优先推荐全城评分高、覆盖品类全的商家，让用户先用起来。

商圈级：差异化分发的关键

商圈级是我觉得最有意思的一层，粒度大概是2-5公里，对应的是一个商圈或者区域。这一层的价值是：不同商圈的用户需求是完全不同的。

我发现，同样是午餐推荐，在金融街推荐商务简餐可能效果更好，但在望京推荐网红餐厅转化率更高；同样是晚上10点，三里屯适合推夜宵烧烤，但回龙观更适合推快餐便利店。

所以商圈级的策略核心是：给每个商圈建立特征画像，然后做差异化分发。这里的关键是要理解商圈的"人群属性"和"消费场景"，而不只是看地理位置。

POI级：精准推荐的最后一公里

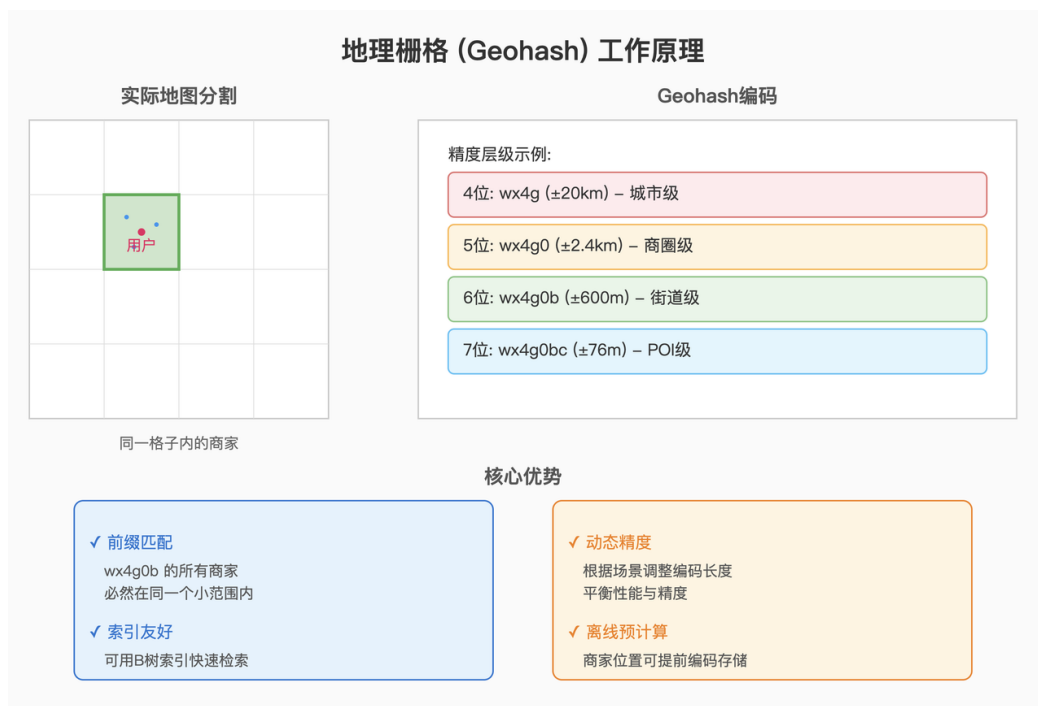
POI（Point of Interest，兴趣点）级是最细粒度，通常是500米到1公里范围。这一层才是我们常说的"附近推荐"，需要实时计算用户和商家的精确距离。

但这里有个很容易踩的坑：精确距离计算成本很高。如果每次用户打开APP，你都要实时算一遍周围所有商家的距离，服务器会被拖垮。这就引出了下一个话题——地理栅栏。

地理栅栏（Geohash）：用空间换时间的设计智慧

地理栅栏是本地生活推荐里最重要的基础设施，但很多PM可能连这个概念都没听过。简单说，它是一种把地球表面划分成网格，然后用编码表示每个格子的技术。

听起来很抽象？我们来看个具体的例子：



地理栅格工作原理

Geohash的核心原理

Geohash的逻辑是：把经纬度坐标编码成一个字符串，比如"wx4g0b"。这个编码有个神奇的特性——相同前缀的编码，地理位置一定相近。

这意味着什么？假设你的位置编码是wx4g0bc，那么所有wx4g0b开头的商家，必然在你附近一定范围内。这样你就不需要实时计算距离了，直接查数据库里"前缀匹配"的商家就行，查询效率提升几十倍。

PM需要关注的三个问题

1. 精度如何选择？

Geohash编码长度决定了精度。4位编码覆盖约20公里，6位覆盖约600米，7位覆盖约76米。不同场景需要不同精度——外卖推荐可能用6位就够了，但共享单车定位需要7位甚至8位。

2. 边界问题怎么处理？

这是个经典的坑。假设用户在wx4g0b和wx4g0c的边界上，如果只查wx4g0b的商家，可能会漏掉对面100米的一家好店。所以实际操作中，需要同时查询周边8个格子的商家，这叫"九宫格查询"。

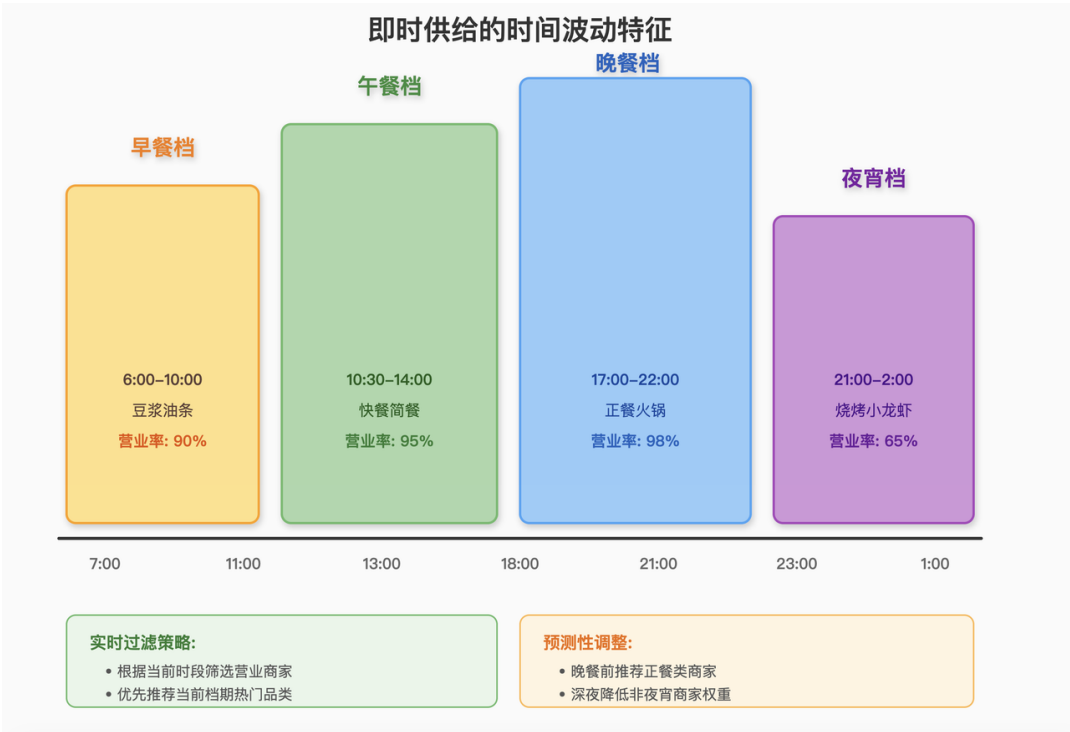
3. 数据如何存储？

这是个工程问题，但PM也要了解。商家的Geohash编码可以离线预计算，存在数据库里建立索引。用户的编码是实时计算的，但计算量很小，几乎没有性能压力。这种"离线预计算+实时轻量查询"的设计，是本地生活推荐能扛住高并发的关键。

即时供给：本地生活的"流动性"挑战

聊完地理，我们聊时间。如果说电商推荐最大的变量是"库存"，那本地生活推荐最大的变量就是"即时供给"。

什么是即时供给？简单说，就是商家的可服务能力是实时变化的。一家餐厅早上7点可能还没营业，中午12点可能爆单接不了新单，晚上10点可能食材卖光了提前打烊。这些状态都是动态的，而且变化频率很高。



即时供给的时间波动

时间维度的三层策略

我观察下来，处理即时供给主要有三个层次：

- 第一层是营业时间过滤：**这个最基础，早上9点不推荐夜宵店，晚上11点不推荐只做午市的餐厅。但实际操作中有很多细节，比如商家设置的营业时间可能不准，或者临时改了时间没更新系统，这时候就需要结合历史订单数据做修正。

- **第二层是实时状态判断：**即使在营业时间内，商家也可能临时打烊、爆单、或者配送运力不足。这需要接入实时数据流，比如最近10分钟的接单率、骑手数量、平均配送时长等。如果一家店最近接单率突然降到50%以下，大概率是出问题了，需要降低推荐权重。
- **第三层是预测性调整：**这是最难的一层。比如现在是下午5点半，用户浏览外卖，你推荐的商家需要考虑：等用户下单、商家接单、制作、配送，送到用户手里可能已经6点半了，这时候用户还想吃午餐类的快餐吗？还是应该推荐晚餐类的正餐？

这里的核心思路是：**不要只看当前时刻，要预测"用户真正消费的时刻"**。这需要建立时间序列模型，预测不同时段、不同品类的需求变化。

供给质量的动态评估

除了时间，供给质量还包括很多其他维度。我之前一直有个困惑：同样距离、同样评分的两家店，为什么有时候推荐A有时候推荐B？

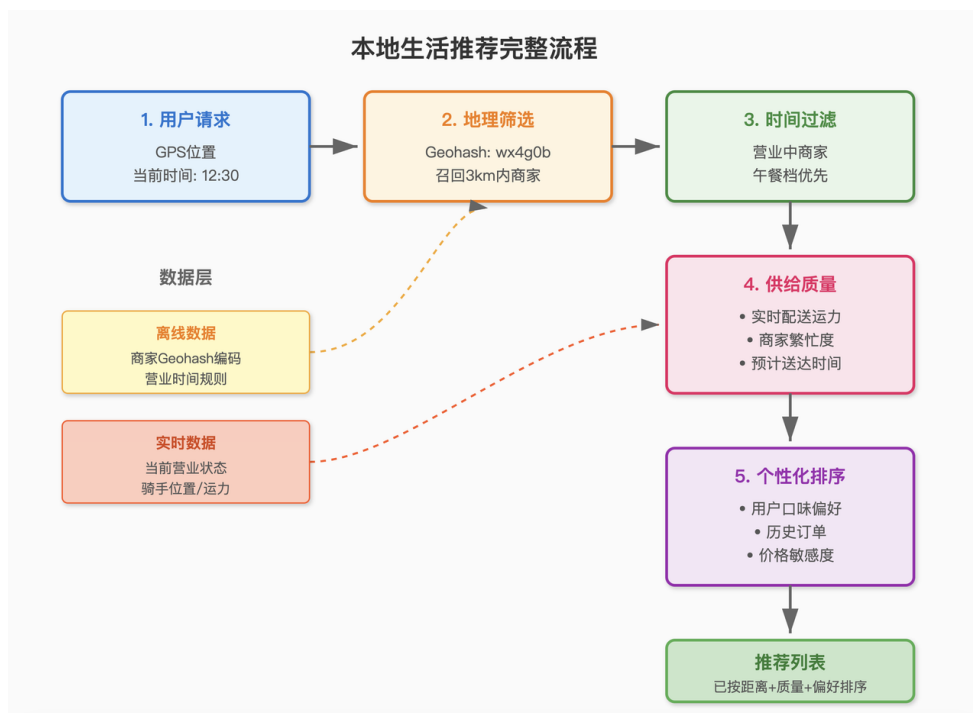
后来我理解了，这里面考虑的是"当下这一刻，哪家店能给用户更好的体验"。具体来说：

- **配送运力：**附近有多少骑手在线？是不是都在配送中？如果运力紧张，即使商家再好也要降权，因为用户等太久会投诉。
- **商家繁忙度：**最近10分钟接了多少单？厨房处理能力如何？如果已经很忙了，再推新单可能导致制作延迟。
- **预计送达时间：**这是个综合指标，结合距离、商家制作速度、骑手配送速度计算。不是简单的物理距离，而是"真实可达时间"。

这些数据都是实时变化的，所以推荐结果也要动态调整。这也是为什么有时候你刷新一下列表，排序就变了——不是系统乱了，而是供给状态变了。

整合：一个完整的本地生活推荐流程

说了这么多，我们把LBS、地理栅栏、即时供给整合起来，看看一次完整的本地生活推荐是怎么跑的：



示例流程

这个流程看起来复杂，但核心逻辑就三句话：

1. 先用Geohash快速圈定候选集，把几万家商家缩小到几百家；
2. 再用时间和供给质量过滤掉不可用的，留下几十家；
3. 最后对这几十家做个性化排序，展示给用户。

说点心里话

虽然我自己没做过本地生活推荐，但研究这个领域的过程中，我最大的感受是：本地生活推荐的难点不在算法，而在对"时空"的理解。

地理位置不是一个静态的坐标，而是用户生活场景的延伸——上班时在CBD，下班后在社区，周末在商场，每个位置对应的需求完全不同。时间也不是简单的小时刻度，而是用户行为模式的体现——工作日午餐求快，周末晚餐求好，深夜想吃的和白天完全两个世界。

作为PM，我们需要把这些"时空场景"转化成可执行的策略。LBS分层、Geohash编码、即时供给评估，这些技术概念背后，其实都是在回答一个问题：怎么在合适的时间、合适的地点，把合适的商家推荐给合适的用户？

如果你也在做本地生活相关的产品，或者对这个领域感兴趣，欢迎留言交流。我虽然不是这个领域的从业者，但通过研究学到的这些思路，或许能给你一些不一样的视角。

